

1. RESEARCH PROBLEM & OBJECTIVE

Refined Research Question (Neutral Scientific Framing)

"To what extent does NFE (Number of Function Evaluations) characterize the computational cost of Neural ODEs, and under what conditions do model capacity, numerical solver algorithms, stiffness, and hardware execution factors cause deviations between NFE and actual wall-clock runtime?"

Research Context: Neural ODEs replace fixed-depth discrete layers with continuous integration: $\frac{dy}{dt} = f_{\theta}(y(t), t)$. While NFE is conventionally treated in machine learning literature as a universal proxy for computational cost, numerical ODE literature demonstrates that actual runtime is governed by a multi-component decomposition:

$$\text{Total Wall-Clock Cost} \times T_{\text{NODE}} = \underbrace{\text{NFE} \times T_{f_{\theta}}}_{\text{Vector Field Evaluation}} + \underbrace{N_{\text{steps}} \times T_{\text{solver_loop}}}_{\text{Tableau / Step Adaptation}} + \underbrace{N_{\text{jev}} \times T_{\text{Jac}}}_{\text{Jacobian}} + \underbrace{N_{\text{lu}} \times T_{\text{LU}}}_{\text{Implicit Linear Algebra / Stiffness}} + \underbrace{T_{\text{GPU-sync}}}_{\text{GPU-synchronization}} + \underbrace{\text{Host-Device Overhead}}_{\text{Data Transfer}}$$

Guiding Rule: We do *not* prematurely invent a new architecture or algorithm. Instead, this phase conducts rigorous literature grounding and controlled benchmarking to characterize when NFE is a faithful metric and when underlying numerical/hardware factors dominate.

2. LITERATURE CORPUS & ARCHITECTURE TAXONOMY (15 PAPERS VERIFIED)

To ensure reproducible grounding, 15 papers across scientific machine learning and numerical analysis were collected, verified, and downloaded directly into a dedicated project archive (`literature/`). They are classified into three structural groups:

Group A: Conventional MLPs6 Papers

Purpose: Identify established vector-field structures to inform the selection of *one fixed architecture*.

- Kim et al. (2021) [Stiff NODEs]
- Finlay et al. (2020) [Regularization]
- Caldana & Hesthaven (2025) [Stiff ROM]
- Chen et al. (2018) [Foundational NODE]
- Ghosh et al. (2020) [STEER]
- Onken & Ruthotto (2020) [Discretize-Opt]

Group B: Modified Architecture3 Papers

Purpose: Analyze how modifying dynamical structure alters per-step cost vs NFE.

- Dupont et al. (2019) [Augmented NODE: lifts state to lower NFE]
- Xia et al. (2021) [Heavy Ball: 2nd order momentum dynamics]
- Poli et al. (2020) [Hypersolvers: auxiliary neural solver acceleration]

Group C: Computational Methods6 Papers

Purpose: Computational profiling, parallel batch solvers, and memory mechanics.

- Lienen & Günnemann (2022) [torchode]
- Rackauckas et al. (2019) [DiffEqFlux.jl]
- Kidger (2022) [PhD Thesis / diffrax]
- Zhuang et al. (2020) [ACA Checkpointing]
- Zhuang et al. (2021) [MALI Reversible]
- Rackauckas et al. (2020) [SciML UDEs]

3. RATIONALE FOR THE TOP-5 PAPER DOWN-SELECTION

The Top 5 papers were selected using a strict criteria-based protocol rather than citation popularity. They collectively satisfy three non-negotiable research requirements:

Required Pillar	Pillar Objective	Assigned Top-5 Papers	Primary Value to Track 2 Research
Pillar 1: Architecture	Ground an established baseline MLP vector field without guessing dimensions.	Kim (2021), Finlay (2020), Caldana (2025)	Confirms 2–3 layer MLPs with 50–64 hidden units and smooth ($\mathcal{O}(1)$) activations ('GELU'/'Tanh') are canonical for ODE integration.
Pillar 2: Computation	Provide concrete timing, solver loop overhead, and NFE metrics.	Finlay (2020), Lienen (2022)	Isolates vector field latency ($T_{f_{\theta}}$) from solver loop overhead (T_{LT}), proving that batched GPU ODEs suffer synchronization overhead.
Pillar 3: Stiffness	Characterize computational cost when explicit solvers fail or become stiff.	Kim (2021), Caldana (2025), Dupont (2019)	Exposes implicit solver mechanics (Jacobians $\# \text{jev}$ and LU factorizations $\# \text{lu}$) where NFE fails as an accurate cost indicator.

4. DEEP-DIVE EXTRACTION MATRIX (TOP-5 PAPERS)

Paper & Citation	Architecture & Activations	Solvers & Tolerances	Systems & Hardware	Computational Metrics	Key Finding & Reusable Pattern
#4 Kim et al. (2021) <i>Chaos (AIP)</i> MIT / SciML	MLP: 1–6 hidden layers, 50 nodes. Activations: Tanh, GELU. Output scaling for concentrations.	Explicit: Tsit5, Dopri5. Implicit: Rosenbrock23, TRBDF2, RadauIIA5. Tol: 10^{-6} to 10^{-12} .	ROBER (stiff 3-species kinetics), POLLU (20-species air pollution). Julia / CPU.	NFE (failed vs successful), loss curves, gradient divergence, numerical error.	Finding: Explicit solvers explode in NFE on stiff systems. Implicit solvers succeed but incur Jacobian & LU overhead. Adapt: Stiff vs non-stiff solver comparison on Van der Pol.
#2 Finlay et al. (2020) <i>ICML 2020</i> McGill / Vector	MLP & CNN: depth 3–6, 64 hidden width. Activation: Softplus. Time t concatenated with state.	Adaptive Dopri5 ('torchdiffeq'), fixed RK4. Tol: 'atol=1e-5', 'rtol=1e-5'.	Density estimation (POWER, GAS, MINIBOOST, MNIST). 4x RTX 2080 Ti GPUs.	NFE curves across epochs, wall-clock training time (hours/days).	Finding: Regularizing dynamics cuts both NFE and wall-clock runtime linearly on fixed models. Adapt: Validate if NFE directly predicts runtime on smooth trajectories.
#10 Lienen (2022) <i>NeurIPS 2022 W.</i> TU Munich	Framework-agnostic ODE wrapper applied to MLPs and GNN dynamics.	Dopri5, Tsit5. Independent step size adaptation within batches.	Van der Pol oscillator, FEN, CNFs. NVIDIA GTX 1080 Ti + Xeon CPU.	Loop Time (\$LT\$) (per-step solver time), Model Time (\$T_f\$), rejected steps.	Finding: Batching forces all samples to step at slowest sample (up to 4x NFE blowup). JIT compilation doubles throughput. Adapt: Profiling loop: $T_{\text{step}} = T_f + T_{\text{solver}}$.
#15 Caldana (2025) <i>Int. J. Numer. Meth. Engng.</i>	Dense FNN: 3–10 layers, 30–100 neurons/layer. Activations: GELU, SiLU, Leaky ReLU.	Implicit: Radau IIA (order 5), BDF. Explicit: RK45. Tol: 10^{-3} to 10^{-8} .	Van der Pol oscillator (μ in [1, 100]), 1D Combustion. Intel CPU workstation.	# fev (NFE), # jev (Jacobian evals), # lu (LU decomp), time (sec).	Finding: Precomputing time transformation converts stiff to non-stiff, cutting NFE & runtime drastically. Adapt: Van der Pol stiffness sweep protocol ($\mu = 1 \dots 100$).
#6 Dupont et al. (2019) <i>NeurIPS 2019</i> Univ. of Oxford	MLP: 1–3 layers, 16–64 width. Activation: ReLU. State augmented with p zero dims: $[z; \mathbf{0}_p]$.	Adaptive Dopri5 ('torchdiffeq'). Standard tolerances.	1D regression, 2D concentric circles, MNIST. NVIDIA GPU / PyTorch.	NFE (forward & backward), test classification error.	Finding: Lifting state into higher dimensions simplifies vector field, dropping NFE from >100 to <30 . Adapt: Decouple state dimension (d) from model capacity (H).

5. EVIDENCE-BASED ARCHITECTURE LOCK (PHASE 4 PROPOSAL)

Guided by the intersection of Groups A and B, we propose to **freeze exactly ONE canonical Neural ODE architecture** for all subsequent computational characterization experiments. This prevents confounding model search with computational profiling:

Proposed Canonical Architecture: Input $y(t) \in \mathbb{R}^d$ [concatenated with t] $\rightarrow$ Linear($d+1 \rightarrow 64$) $\rightarrow$ GELU $\rightarrow$ Linear($64 \rightarrow 64$) $\rightarrow$ GELU $\rightarrow$ Linear($64 \rightarrow d$) $\rightarrow dy/dt$

- Parameter Count:** $\sim 4,500$ parameters (for $d=2$).
- Justification:** 2 hidden layers with width 64 is the exact median of verified architectures in Finlay, Dupont, and Kim. Smooth GELU activation ensures continuously differentiable (C^1) vector fields, enabling stable Jacobian computations for implicit solvers without gradient shattering.

6. CONTROLLED EXPERIMENTAL HIERARCHY & NEXT STEPS

With literature validated and architecture locked, the experimental phase follows a strict *one-variable-at-a-time* protocol:

- Experiment 1 (NFE vs Wall-Clock Tracking):** Fixed architecture, fixed ODE (Van der Pol, $\mu=1$), fixed Dopri5 solver. Vary tolerance (10^{-2} to 10^{-7}) to vary NFE. *Question: Does runtime scale strictly linearly with NFE on CPU vs GPU?*
- Experiment 2 (Model Capacity Effect):** Fixed ODE, fixed solver tolerance. Vary hidden width $H \in \{16, 32, 64, 128, 256\}$. *Question: How does increasing vector field cost T_f alter the slope of NFE vs Runtime?*
- Experiment 3 (Stiffness Dynamics):** Fixed architecture. Sweep Van der Pol stiffness $\mu \in \{1, 5, 10, 25, 50, 100\}$. *Question: When does explicit Dopri5 undergo NFE explosion, and does loop time (\$LT\$) increase?*
- Experiment 4 (Solver Type & Overhead Breakdown):** Compare explicit (Dopri5, Tsit5) vs implicit (Radau, Rosenbrock) on identical weights. Measure NFE vs Jacobian evaluations ('# jev') and LU factorizations ('# lu').